

Mini Project 1 Report

Antara Mazumdar, Ammara Hussain, Naomi Francis



Figure 1: Our working model!

Introduction

Through this project, we aimed to learn how to program an Arduino R3 board to control 5 LEDs incorporating different modes via digital and analog input as well as building an encasing for it. The use of the final product is a bike light, and with this final use in mind we had goals of relevant modes and user accessibility. Our goal of relevant modes required us to learn about programming in Arduino, and our goal of accessibility for users required us to design an easy to use case for bikers, pushing us to learn more about CAD and model fabrication.

Initial Setup

The Arduino Uno R3

The microcontroller that we used for this project was an Arduino Uno R3. The most useful parts of this microcontroller were the Pulse Width Modulation pins and the compatibility of the software and controller with our computers. The Pulse Width Modulation allowed us to change the brightness of the LEDs rather than just setting them to on or off. Because we all have Arduino software on our computers, we could each write different parts of the code and test it out.

Push Button

We used a 2-pin push button to switch between modes when pressed. We used a pull down resistor of $10\text{k}\Omega$ for the push button to mitigate floating voltage across the push button when it is in the off state. Floating voltage can be caused by stray electromagnetic fields and other environmental sources, so this high resistance pull down resistor brings any floating voltage close to 0V so that a push is not registered.

Choosing Resistor Values

To start off, we connected our 5 LEDs to the Arduino board using resistors of appropriate values. To do this, we looked at the datasheets of each LED, selecting a reasonable operating current and calculating the required resistor values based on that. In general, we use the following equation to obtain the values for the resistances for each LED -

$$R = \frac{V - V_{LED}}{i}$$

Figure 2 (below) shows the calculated required resistances. Since some of the values obtained were not within the standard resistors, we chose the nearest appropriate resistor. This did not affect our circuit or effectiveness in any way because we also took some margins on the operating current.

<u>Red LED</u> $i = 25\text{ mA}$ $\therefore R_R = \frac{V - V_{LED}}{i} = \frac{5 - 2}{0.025} = 120\Omega$	<u>Green LED</u> $i = 15\text{ mA}$ $R_G = \frac{V - V_{LED}}{i} = \frac{5 - 2}{0.015} = 200\Omega$
<u>Yellow LED</u> $i = 20\text{ mA}$ $\therefore R_Y = \frac{V - V_{LED}}{i} = \frac{5 - 2}{0.02} = 150\Omega$	<u>Blue LED</u> $i = 25\text{ mA}$ $R_B = \frac{V - V_{LED}}{i} = \frac{5 - 3}{0.025} = 80\Omega$

Figure 2: Calculations for resistor values for the LEDs

The diagram shows an Arduino Uno R3 microcontroller board connected to a 7-segment display. The display is configured for common anode, with each segment connected to a specific Arduino pin through a current-limiting resistor. The segments are labeled: D1 (RED), D2 (YELLOW), D3 (GREEN), D4 (BLUE1), and D5 (BLUE2). A potentiometer (RV1) is connected to the A0 pin for trim adjustment. A push button (SW1) is connected to the D1/TX pin for control. The circuit is powered by VCC and GND.

LED modes

- 1) All on, adjustable brightness
- 2) Red, yellow, green like in a traffic light, with the blues always on
- 3) Left blue blinking
- 4) Right blue blinking
- 5) Both blues blinking at the same time
- 6) All off

Part 1 - Digital Input

Setup

To set up the program, we had to indicate whether the components were “input” or “output”. All of the LEDs were set as “output”, as shown below, because the microcontroller is not reading anything from this component.

```
void setup() {  
  pinMode(REDA, OUTPUT);  
  pinMode(YELLOW, OUTPUT);  
  pinMode(GREEN, OUTPUT);  
  pinMode(BLUE1, OUTPUT);  
  pinMode(BLUE2, OUTPUT);
```

Figure 4: 5 LEDs set to “output” in setup

The components that the microcontroller “reads”, however, such as the position of the potentiometer or the status of the button, need to be set as “input”:

```
  pinMode(BUTTON, INPUT);  
  pinMode(POTENTIOMETER, INPUT);
```

Figure 5: Push button and potentiometer set to “input” in setup

Another piece of the setup is writing the function that adjusts the brightness for the LEDs for the mode that features adjustable brightness with analog potentiometer input. This function takes the integer “brightness”, which comes from the potentiometer reading, and applies it to the LEDs.

```
void setBrightnessForAllLEDs (int brightness) {  
  analogWrite(REDA, brightness);  
  analogWrite(YELLOW, brightness);
```

Figure 6: Brightness written for red and yellow LEDs in the “setBrightnessForAllLEDs” function

Loop

The loop function checks for button presses and then waits to prevent bounce before calling the runModes() function. Debouncing is important because otherwise the button push will sometimes register as two pushes happening right after each other. In each of our modes, we incorporated break statements so that every time a certain action was performed, it would go back to check if the button was pressed and then move to the next one.

```

void loop() {
  handleButtonPress();
  unsigned long currentTime = millis();
  if (currentTime - previousTime >= timeDelay) { // handles time delays without using delay()
    previousTime = currentTime;
    runModes();
  }
}

```

Figure 7: loop() function that runs forever

Mode 1

Mode 1 features all LEDs on, with the option to adjust them with the potentiometer. By using “digitalWrite”, we set each LED to “HIGH”, which means it is receiving a full 5V from the Arduino and is therefore turned on. The integer “potentiometerValue” reads a value from the potentiometer, and the integer “brightness” takes that value and converts it to the Arduino Pulse Width Modulation scale, which goes from 0-255.

```

void allOn() { // turns all the LEDs on
  digitalWrite(RED, HIGH);
  digitalWrite(YELLOW, HIGH);
  digitalWrite(GREEN, HIGH);
  digitalWrite(BLUE1, HIGH);
  digitalWrite(BLUE2, HIGH);

  int potentiometerValue = analogRead(POTENTIOMETER); // reads value of potentiometer
  int brightness = map(potentiometerValue, 0, 1023, 1, 255); // converts the potentiometer range to the arduino brightness range
  setBrightnessForAllLEDs(brightness); // uses brightness function to set the brightness for all LEDs
}

```

Figure 8: Code snippet showing function allOn() used to turn all the LEDs on and add controls for potentiometer dimming

Mode 2

In Mode 2, the bottom lights of the bike light illuminate in the order red, yellow, green while the top blue LEDs stay on. Under “case #”, the LEDs are assigned to be either on or off so that they can turn on in sequence. We chose to implement this mode using step and switch instead of a single code block so that we could handle changing modes (button press) between different steps.

```

void trafficLights() { //simulates a traffic light

  switch (step) {
    case 0: // red is on, yellow and green are off
      digitalWrite(REDF, HIGH);
      digitalWrite(YELLOW, LOW);
      digitalWrite(GREEN, LOW);
      break;
    case 1: // yellow is on, red and green are off
      digitalWrite(REDF, LOW);
      digitalWrite(YELLOW, HIGH);
      digitalWrite(GREEN, LOW);
      break;
    case 2: // green is on, red and yellow are off
      digitalWrite(REDF, LOW);
      digitalWrite(YELLOW, LOW);
      digitalWrite(GREEN, HIGH);
      break;
    digitalWrite(BLUE1, HIGH);
    digitalWrite(BLUE2, HIGH);
  }

  step++;
  if (step > 2) step = 0;
}

```

Figure 9: Code snippet showing function trafficLights that turns red, yellow, green on in order while both blues stay continuously on

Mode 3

Mode 3 imitates the left turn signal for a vehicle. This function switches “BLUE1”, our left LED, between on and off.

```

void leftBlink() { //simulates a left turn indicator

  switch (step) {
    case 0: // left blue turned on
      digitalWrite(BLUE1, HIGH);
      break;
    case 1: // left blue turned off
      digitalWrite(BLUE1, LOW);
      break;
  }

  step++;
  if (step > 1) step = 0;
}

```

Figure 10: Code snippet showing function leftBlink() that causes the left blue LED to blink

Mode 4

Similar to Mode 3, Mode 4 imitates the right turn signal for a vehicle, now switching “BLUE2” between off and on.

```

void rightBlink() { //simulates a right turn indicator

  switch (step) {
    case 0: // right blue is turned on
      digitalWrite(BLUE2, HIGH);
      break;
    case 1: // right blue is turned off
      digitalWrite(BLUE2, LOW);
      break;
  }

  step++;
  if (step > 1) step = 0;
}

```

Figure 11: Code snippet showing function rightBlink() that causes the right blue LED to blink.

Mode 5

The final mode, Mode 5, imitates hazard lights on a vehicle. This function flashes both blue LEDs using the same method as the prior two modes.

```

void hazardLights() { //simulates hazard lights

  switch (step) {
    case 0: // both blues are turned on
      digitalWrite(BLUE1, HIGH);
      digitalWrite(BLUE2, HIGH);
      break;
    case 1: // both blues are turned off
      digitalWrite(BLUE1, LOW);
      digitalWrite(BLUE2, LOW);
      break;
  }

  step++;
  if (step > 1) step = 0;
}

```

Figure 12: Code snippet showing function hazardLights() that makes both blue lights blink

Part 2 - Analog Input

To add an analog component to the bike light, we decided to use a trimming potentiometer that allows the user to easily dim or increase the brightness of the LEDs in the first mode. A potentiometer has three pins, with the two outer ones connecting to ground and power, and the middle connecting to an “Analog In” pin on the microcontroller. In this instance, our power source was the 5V line of the solderless breadboard, which connects to the 5V pin of the microcontroller. The microcontroller gets its power from the laptop, where it is connected with a USB port. The knob of the potentiometer, which is turned by the user, acts as a wiper across an internal resistor within the potentiometer. When the user turns the knob, the wiper moves, changing the resistance. The resistance range of the potentiometer that we used goes from 10

ohms to 2 megaohms. By setting the potentiometer as an input, the microcontroller can read measurements from it. This allowed us to read the position of the potentiometer dial through the microcontroller and adjust the brightness accordingly.

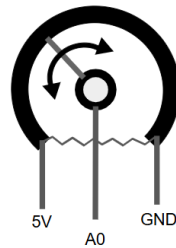


Figure 13: A potentiometer diagram, showing how resistance changes with analog input and pin connections

```
// Brightness Function
// Takes variable brightness as determined by the potentiometer
void setBrightnessForAllLEDs (int brightness) {
  analogWrite(RED, brightness);
  analogWrite(YELLOW, brightness);
  analogWrite(GREEN, brightness);
  analogWrite(BLUE1, brightness);
  analogWrite(BLUE2, brightness);
}
```

Figure 14: Code snippet with setBrightnessForAllLEDs() that takes in brightness as a value from the potentiometer and adjusts the LED brightnesses accordingly

Note that unlike the previous input method which used digitalWrite, this uses analogWrite because it is an analog input that the code then converts into values that the Arduino can read.

Part 3 - Mechanical Housing

Layout

In our initial planning for the mechanical housing, we decided to keep our design compact in order for our bike light to mount easily and have little interference with the function of the bike. As such, we decided to stack the breadboard on top of the arduino in order to save space. To make this design we divided the housing into three main sections:

- 1) Arduino containment (bottom) - contains the arduino and wire connections from the breadboard

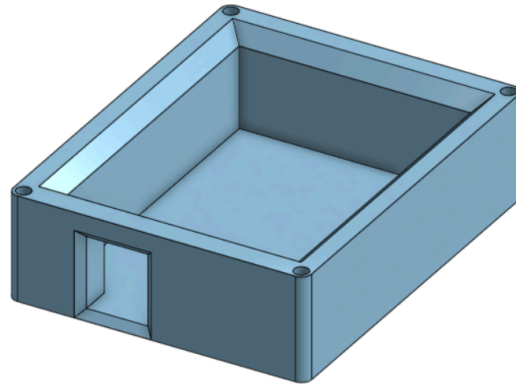


Figure 15: CAD of bottom Arduino housing

We designed the Arduino casing to snugly hold the arduino with little movement. We included a port hole in the housing design to allow external connection to the Arduino. Additionally, we added small holes in each corner on the top face of the containment to allow the pegs at the bottom of the breadboard loft to sit into, in order to ensure alignment.

- 2) Breadboard loft (middle) - dedicated to holding the breadboard securely above the arduino and directing wiring to the arduino

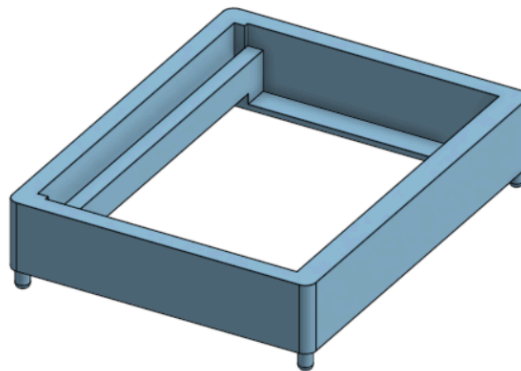


Figure 16: CAD of middle breadboard loft

We wanted the breadboard to be very stable so that the LEDs, button, and potentiometer would not move. In order to make sure that the components stayed in their place, we designed the breadboard loft to hold our breadboard exactly. The breadboard sits on a ledge, with open space underneath to both reduce material usage and provide extra wiggle room for any wire beneath. We also designed a slot on one side of the section to fit wiring, with additional spacing provided

under the wall separating the wire slot and breadboard, to route wires to the Arduino. We left additional spacing above the breadboard to allow components and wiring to sit flush without interference. On the bottom, we created pegs that would nestle in the holes of the Arduino casing to maintain alignment.

- 3) Housing front panel/housing connector (top) - encases the breadboard and arduino as well as provides a front panel to the bike light where LEDs and buttons are nestled

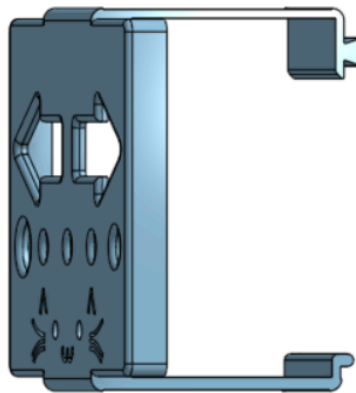


Figure 17: CAD of front plate/circuit casing lid from the side to show panel clips

To hold all the components of the mechanical housing together, we designed a piece that would go above and around the two stacked components housing the arduino and breadboard. We added thin side panels with clips at each end to hold the case together securely. The side panels were given enough thickness to provide strength yet retain slight flexibility for assembly so the top piece could be added either by being slid on or by being placed from top down. For mounting purposes, we designed the top panel to include a dovetail joint to slide into the mounting component we designed. This was to allow simple attachment and removal from the bike.

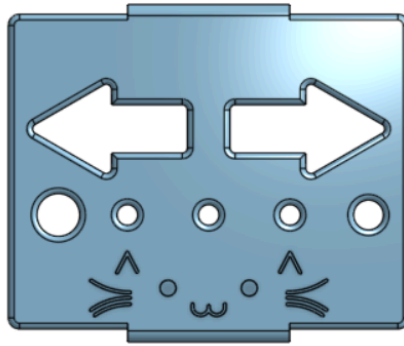


Figure 18: CAD of front plate/circuit casing lid from a head on perspective

On the top face of this piece, we added all necessary holes for the LEDs, button, and potentiometer. We made the LED holes a size that would allow the LEDs to be suspended halfway for an enhanced visual appearance. We also included arrow holes so the arrows could be raised above the surface of the front panel. This is to enable the light from the blue LEDs below to diffuse properly and illuminate the full shape. Lastly, we ensured that the hole for the button and potentiometer were in line with the holes for the red, yellow, and green LEDs to allow an easy fit based on the circuit layout.

Mounting to the Bike

We determined that the best place to mount the bike light would be on the back of the basket of the Olin bikes. Each basket already has a mounting point used for a safety reflector. Since the bike light serves the same purpose and more, we thought that we would design our mount to utilize this attachment point. Taking note of the dimensions, we designed a removable mount that screws onto the bike basket and slides onto the bike light. The front side has the other end of the dovetail joint, allowing for easy attachment and detachment of the LED box.

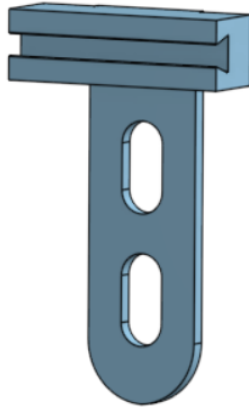


Figure 19: CAD of bike light mounting piece



Figure 20: Bike light mounting piece in place of safety reflector

Another thing we took into consideration was the power supply. Currently, our circuit is set up to take a 5V input. However, we did not have a 5V battery, so we designed a battery holder attachment that houses one 9V battery that connects to the circuit. It is designed to hook into the port hole already existing on the Arduino containment piece. If we were to use the battery in place of a cable attached to the computer, then we would need to change some components within the circuit.

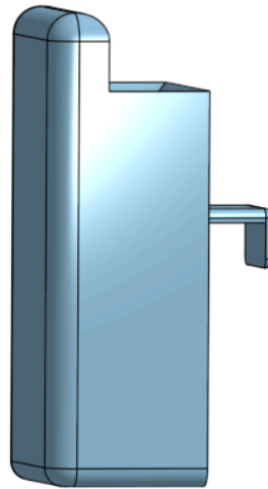


Figure 21: CAD of battery holder that attaches to the side of the bike light

Aesthetics

In addition to focusing on the functionality of our signaling system, we also placed emphasis on aesthetics to make our design more appealing. Instead of exposing raw LEDs, we used backlit arrows that protruded outward, which gave the blinker settings a more polished look. For the other LEDs which were exposed, we ensured they were only halfway visible and flush with the surface of our casing. We also designed caps for both the button and potentiometer which made each piece larger so it was easy to push or turn. As for the circuit containment itself, we made sure the outer shell was kept thin to maintain a sleek profile so it wasn't too bulky when mounted. Since we had extra space below the LEDs, we included a happy cat design which represented the spirit of our team.



Figure 22: Final bike light mounted on bike basket

Reflection

Throughout this project, we learned about the importance of communication between team members working on coding, electrical design, and mechanical design. We made many models of the physical casing, which likely could have been decreased with more communication. Some improvements that we can implement with more time spent on the project include applying the brightness adjustments to all modes, so that the user can change how bright the lights are based on time of day or preference. Another improvement that we could implement is adjusting our code and electrical components configuration to receive power from a 9V battery. We created a holder for the battery to hang from the side of the bike light, so if we spent more time on the project, we could alter the code and electrical parts to handle 9V as well. A final improvement for our bike light would be adding space for the wires on both sides of the breadboard in the housing for the breadboard. Our current design only has space for the wires coming from the breadboard to connect to the microcontroller on one side, making putting together the light difficult.

Demo Video

<https://drive.google.com/file/d/1shDcNrc6Soo76p06pQkpgDJGU8ePbHVi/view?usp=sharing>

Sources

“Arduino Potentiometer with Multiple LEDs [Tutorial].” *The Robotics Back-End*, 1 Nov. 2021, <https://roboticsbackend.com/arduino-potentiometer-with-multiple-leds-tutorial/>.

“Pull Up/Down Resistors” *YouTube*. <https://www.youtube.com/watch?v=ZA4gnrVuE8g&t=3s>
Accessed 11 Sep. 2025.

Code Appendix

```
// Pin assignments
const int RED = 11;
const int YELLOW = 9;
const int GREEN = 5;
const int BLUE1 = 10;
const int BLUE2 = 6;
const int BUTTON = 2;
const int POTENTIOMETER = A0;

// Mode control
int mode = 0;
unsigned long lastDebounceTime = 0;
const unsigned long debounceDelay = 50;
int lastButtonState = HIGH; // HIGH is off for this button
int buttonState = HIGH; // HIGH is off for this button

// Timing for LED updates
unsigned long previousTime = 0;
const unsigned long timeDelay = 500; // 1 second delay for traffic light
int step = 0;

void setup() {
  pinMode(RED, OUTPUT);
  pinMode(YELLOW, OUTPUT);
  pinMode(GREEN, OUTPUT);
  pinMode(BLUE1, OUTPUT);
  pinMode(BLUE2, OUTPUT);
  pinMode(BUTTON, INPUT);
  pinMode(POTENTIOMETER, INPUT);
}
```

```

// Brightness Function
// Takes variable brightness as determined by the potentiometer
void setBrightnessForAllLEDs (int brightness) {
    analogWrite(RED, brightness);
    analogWrite(YELLOW, brightness);
    analogWrite(GREEN, brightness);
    analogWrite(BLUE1, brightness);
    analogWrite(BLUE2, brightness);
}

void loop() {
    handleButtonPress();
    unsigned long currentTime = millis();
    if (currentTime - previousTime >= timeDelay) {    // handles time delays
without using delay()
        previousTime = currentTime;
        runModes();
    }
}

void handleButtonPress() {    // checks for button press
    int reading = digitalRead(BUTTON);

    if (reading != lastButtonState) {
        lastDebounceTime = millis();
    }

    if ((millis() - lastDebounceTime) > debounceDelay) {    // prevents long
button presses from being read as multiple presses
        if (reading != buttonState) {    // checks that button was pressed
            buttonState = reading;
            if (buttonState == LOW) {    // increments mode if button is pressed
                mode = (mode + 1) % 6;
                step = 0;
                allOff(); // turns the LEDs off at the end of each mode
            }
        }
    }

    lastButtonState = reading;
}

void runModes() {    // calls the modes based on the value of the mode variable
    switch (mode) {
        case 0: allOff(); break;
        case 1: allOn(); break;

```

```

        case 2: trafficLights(); break;
        case 3: leftBlink(); break;
        case 4: rightBlink(); break;
        case 5: hazardLights(); break;
    }
}

void allOn() { // turns all the LEDs on
    digitalWrite(RED, HIGH);
    digitalWrite(YELLOW, HIGH);
    digitalWrite(GREEN, HIGH);
    digitalWrite(BLUE1, HIGH);
    digitalWrite(BLUE2, HIGH);

    int potentiometerValue = analogRead(POTENTIOMETER); // reads value of
    potentiometer
    int brightness = map(potentiometerValue, 0, 1023, 1, 255); // converts the
    potentiometer range to the arduino brightness range
    setBrightnessForAllLEDs(brightness); // uses brightness function to set the
    brightness for all LEDs
}

void allOff() { // turns all the LEDs off
    digitalWrite(RED, LOW);
    digitalWrite(YELLOW, LOW);
    digitalWrite(GREEN, LOW);
    digitalWrite(BLUE1, LOW);
    digitalWrite(BLUE2, LOW);
}

void trafficLights() { //simulates a traffic light
    digitalWrite(BLUE1, HIGH);
    digitalWrite(BLUE2, HIGH);
    switch (step) {
        case 0: // red is on, yellow and green are off
            digitalWrite(RED, HIGH);
            digitalWrite(YELLOW, LOW);
            digitalWrite(GREEN, LOW);
            break;
        case 1: // yellow is on, red and green are off
            digitalWrite(RED, LOW);
            digitalWrite(YELLOW, HIGH);
            digitalWrite(GREEN, LOW);
            break;
        case 2: // green is on, red and yellow are off
            digitalWrite(RED, LOW);
            digitalWrite(YELLOW, LOW);
            digitalWrite(GREEN, HIGH);
    }
}

```

```

        break;
    }

    step++;
    if (step > 2) step = 0;
}

void leftBlink() {    //simulates a left turn indicator

    switch (step) {
        case 0:    // left blue turned on
            digitalWrite(BLUE1, HIGH);
            break;
        case 1:    // left blue turned off
            digitalWrite(BLUE1, LOW);
            break;
    }

    step++;
    if (step > 1) step = 0;
}

void rightBlink() {    //simulates a right turn indicator

    switch (step) {
        case 0:    // right blue is turned on
            digitalWrite(BLUE2, HIGH);
            break;
        case 1:    // right blue is turned off
            digitalWrite(BLUE2, LOW);
            break;
    }

    step++;
    if (step > 1) step = 0;
}

void hazardLights() {    //simulates hazard lights

    switch (step) {
        case 0:    // both blues are turned on
            digitalWrite(BLUE1, HIGH);
            digitalWrite(BLUE2, HIGH);
            break;
        case 1:    // both blues are turned off
            digitalWrite(BLUE1, LOW);
            digitalWrite(BLUE2, LOW);
            break;
    }
}

```

```
    step++;  
    if (step > 1) step = 0;  
}
```